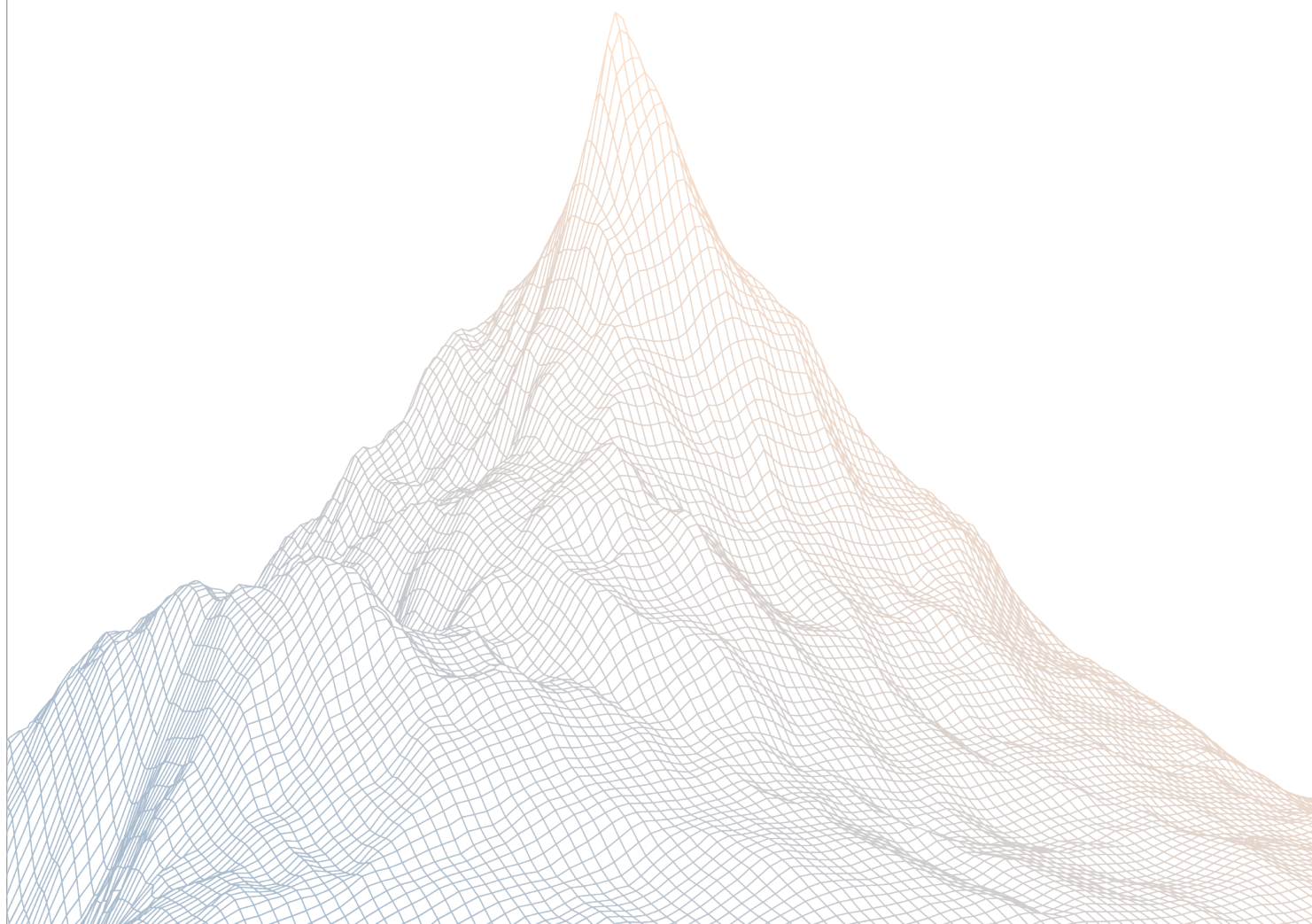


Nad.fun

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: April 22nd to April 28th, 2026
AUDITED BY: ast3ros
said

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Nad.fun	4
2.2	Scope	4
2.3	Audit Timeline	6
2.4	Issues Found	6
3	Findings Summary	6
4	Findings	8
4.1	Medium Risk	9
4.2	Low Risk	18
4.3	Informational	42

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Nad.fun

Welcome to Nad.fun! We're redefining the memecoin experience by combining token creation, trading, and social engagement in one seamless platform.

2.2 Scope

The engagement involved a review of the following targets:

Target	nadfun-contract-v2
Repository	https://github.com/Naddotfun/nadfun-contract-v2.git
Commit Hash	65f82d755582f970f8d992c46a3e567f231462ff
Files	core/BondingCurve.sol core/FeeCollector.sol core/ProtocolManager.sol core/CreatorFeeProcessor.sol core/TokenRegistry.sol core/LPManager.sol dex/NadFunPair.sol dex/NadFunFactory.sol

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/Naddotfun/nadfun-contract-v2.git
-------------------	---

Commit Hash	fad1d13179ae103f263c163ff3b43789b95f66e4
--------------------	--

Files	<code>core/BondingCurve.sol</code> <code>core/FeeCollector.sol</code> <code>core/ProtocolManager.sol</code> <code>core/CreatorFeeProcessor.sol</code> <code>core/TokenRegistry.sol</code> <code>core/LPManager.sol</code> <code>dex/NadFunPair.sol</code> <code>dex/NadFunFactory.sol</code>
--------------	---

2.3 Audit Timeline

April 22, 2026	Audit start
April 28, 2026	Audit end
May 9, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	5
Low Risk	13
Informational	13
Total Issues	31

3

Findings Summary

ID	Description	Status
M-1	Fee distribution mismatch due to stale cached rate in FeeCollector	Resolved
M-2	Flash swap fee double charged on quote token	Acknowledged
M-3	Excess tokens are transferred to fee receiver instead of being burned	Acknowledged
M-4	CreatorFeeProcessor afterDeposit try/catch leaves quote tokens stranded at vault	Resolved
M-5	Buyer overpays fees on the refunded excess portion at the curve cap	Acknowledged
L-1	buy reverts instead of charging fee when totalFeeRate >= BPS	Resolved
L-2	Creator fee is charged in initial buy	Acknowledged
L-3	Protocol does not support fee-on-transfer quote tokens	Acknowledged
L-4	virtualTokenReserve is not bounded	Resolved
L-5	No slippage protection in vault swaps during settlement	Resolved
L-6	getAmountIn does not enforce exact out semantics	Resolved
L-7	Graduation can revert if graduateFee is increased post creation	Resolved
L-8	Incorrect seller in Sell Event	Resolved
L-9	Duplicate vault addresses are not rejected in _setupVaults	Resolved
L-10	BondingCurve _calculateFees zeroes amountAfterFees when totalFeeRate >= BPS, even if snipingFeeRate + protocolFeeRate < BPS	Resolved
L-11	NadFunFactory.createPair permissionless, pre-deploy DOS for predicted salt	Acknowledged

ID	Description	Status
L-12	BondingCurve setModule enables retroactive re-routing of all fees	Resolved
L-13	BondingCurve _calculateFees else-branch zeroes creator fee, but FeeCollector re-splits via creator-rate share	Resolved
I-1	Missing aggregate fee cap can brick NadFunPair quoting and DEX swap execution	Resolved
I-2	Direct use of BondingCurve.buy and BondingCurve.sell exposes users to sandwich attacks	Resolved
I-3	Misleading comment in BondingCurveLibrary	Resolved
I-4	Token address is not validated in getAmountIn and getAmountOut	Resolved
I-5	_mintFee preserves a 1/6 LP-fee share despite the reduced 0.25% LP fee	Resolved
I-6	getAmountOut silently returns 0 instead of reverting	Resolved
I-7	_calculateFees can revert when snipingFeeRate exceeds 100%	Resolved
I-8	Dead code in _calculateFees	Resolved
I-9	BondingCurve getAmountOut / getAmountIn continue to quote after graduation	Resolved
I-10	Missing zero address check for MODULE_CREATOR_FEE_PROCESSOR	Resolved
I-11	NadFunFactory.setImplementation/setFeeTo and FeeCollector.setCurveProtocolFeeRate/setDexProtocolFeeRate emit no events	Resolved
I-12	Donation before create() could cause third-party reputational and market-signal manipulation	Resolved
I-13	addQuoteToken and updateQuoteToken accept zero or inverted reserves	Resolved

4

Findings

4.1 Medium Risk

A total of 5 medium risk findings were identified.

[M-1] Fee distribution mismatch due to stale cached rate in FeeCollector

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [FeeCollector.sol#L100-L102](#)
- [BondingCurve.sol#L430](#)

Description:

BondingCurve._calculateFees reads the live protocol fee rate from ProtocolManager when computing fees:

```
function _calculateFees(address token, uint256 amount,
Curve storage curve, bool withSniping)
    internal
    view
    returns (uint256 protocolFee, uint256 snipingFee,
uint256 creatorFee, uint256 amountAfterFees)
{
    ...
    uint256 snipingFeeRate = withSniping ? _getSnipingFeeRate(token) :
0;
    ▶ uint256 protocolFeeRate
= _protocolManager.curveProtocolFeeRate(curve.quoteToken);
    uint256 creatorFeeRate = curve.creatorFeeRate;
    uint256 totalFeeRate = snipingFeeRate + protocolFeeRate
+ creatorFeeRate;
    ...
}
```

However, FeeCollector.collectFee distributes the collected fees using the rate cached at

token creation time (config.curveProtocolFeeRate):

```
function collectFee(address pair) external {  
    ...  
    uint16 activeProtocolFeeRate =  
    msg.sender == _bondingCurve ? config.curveProtocolFeeRate :  
    config.dexProtocolFeeRate;  
    uint256 combinedFeeRate = uint256(config.creatorFeeRate)  
    + uint256(activeProtocolFeeRate);  
    ...  
}
```

If updateQuoteToken changes the protocol fee rate after a token is created, the two rates diverge. The fee charged to the user reflects the new live rate, but the fee distributed to the protocol reflects the old cached rate. This causes the protocol to either over or under receive its share, with the difference silently misallocated to the creator.

Recommendations:

FeeCollector.collectFee should query the live protocol fee rate from ProtocolManager rather than relying on the cached value stored at token creation.

Nad.fun: Resolved with [@6fc0786973 ...](#)

Zenith: Verified.

[M-2] Flash swap fee double charged on quote token

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [NadFunPair.sol#L266-L293](#)

Description:

`_collectFee` independently checks whether the quote token appears as input (`quoteIn > 0`) and as output (`quoteOut > 0`), accumulating a fee for each. In a normal buy or sell only one branch fires. However, a flash swap where the quote token is both sent out and returned causes both branches to trigger, charging the protocol fee twice on the same swap.

```
function _collectFee(uint256 amount0In, uint256 amount1In,
uint256 amount0Out, uint256 amount1Out) private {
    address feeCollectorAddr = feeCollector;
    try IFeeCollector(feeCollectorAddr).getFeeConfig(address(this))
returns (
    IFeeCollector.FeeConfig memory config
) {
    ...
    uint256 quoteIn = quoteIsToken0 ? amount0In : amount1In;
    if (quoteIn > 0) {
        swapFee += quoteIn * uint256(feeRate) / BPS;
    }

    uint256 quoteOut = quoteIsToken0 ? amount0Out : amount1Out;
    if (quoteOut > 0) {
        swapFee += quoteOut * uint256(feeRate) / (BPS - LP_FEE_RATE
- feeRate);
    }

    ...
    } catch {}
}
```

When `quoteIn > 0` and `quoteOut > 0` simultaneously, `swapFee` is incremented by both the

buy-side and sell-side fee calculations, making the flash swap disproportionately expensive and inconsistent with the intended single-direction fee model.

Recommendations:

- Define a separate explicit fee rule for that case OR
- Revert when `quoteIn > 0 && quoteOut > 0` if flash swaps aren't intended.

Nad.fun: Acknowledged.

This behavior is intentional for non-standard flash-style swaps where the quote token appears as both input and output in the same pair swap. Normal router/adaptor buy and sell paths only activate one quote-side fee branch. For this unsupported quote-in plus quote-out path, the pair applies creator/protocol fees to both quote movements instead of netting them. Callers that intentionally use this path must account for both fee branches when choosing repayment and output amounts. No code change is planned.

[M-3] Excess tokens are transferred to fee receiver instead of being burned

SEVERITY: Medium

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: High

Target

- [BondingCurve.sol#L404-L406](#)

Description:

The protocol documentation states that: Excess token burn (DEX price = curve price matching). It's to ensure DEX price matches the bonding curve price. However, `_graduateV1` instead transfers excess tokens to the `feeReceiver`, directly violating this policy.

This can lead to:

- The fee receiver receives tokens that were supposed to be burned. They can immediately dump these tokens on the AMM, disrupting the intended DEX/curve price parity at graduation.
- The protocol's documented token supply model is not accurately implemented, as excess supply enters circulation rather than being permanently removed.

```
function _graduateV1(address token, Curve storage curve) internal {
    ...
    {
        uint256 currentTokenBalance
        = IERC20(token).balanceOf(address(this));
        uint256 excessToken = currentTokenBalance - tokenForLiquidity;
        if (excessToken > 0) {
            IERC20(token).safeTransfer(_protocolManager.feeReceiver(),
            excessToken);
        }
    }
    ...
}
```

Recommendations:

Transfer excess tokens to burn address instead.

Nad.Fun: Acknowledged.

This behavior is intentional. We currently send any graduation excess token balance to the protocol fee receiver rather than burning it.

Operationally, we are tuning the virtual reserve parameters so that the graduation path leaves little to no excess token balance in normal configurations. If any residual excess remains due to rounding or parameterization, it is treated as protocol-owned inventory and transferred to the fee receiver.

We understand the documentation describes this as an excess-token burn policy, so we will align the documentation/spec wording with the implemented behavior rather than changing the transfer destination here.

[M-4] CreatorFeeProcessor afterDeposit try/catch leaves quote tokens stranded at vault

SEVERITY: Medium

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Low

Target

- [CreatorFeeProcessor.sol#L74-L83](#)

Description:

```
// ...
        if (amount > 0) {
            address vaultAddr = vaults[i].vault;
            IERC20(quoteToken).safeTransfer(vaultAddr, amount);
            emit Distribute(token, vaultAddr, amount);

            try IVault(vaultAddr).afterDeposit(token, quoteToken,
amount) {}
            catch (bytes memory reason) {
                emit CallbackFail(vaultAddr, amount, reason);
            }
        }
// ...
```

If afterDeposit reverts (e.g., ProtocolHalted during emergency halt + vault hook attempting buyback), the safeTransfer is outside the try block and is not rolled back. The amount arrives at the vault, no recovery function exists in CreatorFeeProcessor.

Recommendations:

Consider add an admin callable rescue function.

Nad.fun: Resolved with [@2a4d02239c ...](#)

Zenith: Verified.

[M-5] Buyer overpays fees on the refunded excess portion at the curve cap

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [BondingCurve.sol#L253-L256](#)
- [BondingCurve.sol#L292-L295](#)

Description:

When a buy crosses the graduation cap, fees are deducted on the FULL quoteIn, then the trade clamps to availableTokenOut. The unused excessQuoteIn = quoteInAfterFees - requiredQuoteIn is transferred to feeReceiver (NOT refunded to the buyer):

```
// ...
(uint256 protocolFee, uint256 snipingFee, uint256 creatorFee,
 uint256 quoteInAfterFees) =
  _calculateFees(token, quoteIn, curve, true);
// ...
if (tokenOut > availableTokenOut) {
  // ...
  > uint256 excessQuoteIn = quoteInAfterFees - requiredQuoteIn;
  if (excessQuoteIn > 0) {
    > IERC20(curve.quoteToken).safeTransfer(_protocolManager.feeReceiver(),
      excessQuoteIn);
  }
  quoteInAfterFees = requiredQuoteIn;
}
// ...
```

The fees corresponding to the refunded portion are NOT recalculated and refunded. The buyer pays sniping + protocol + creator fees on the FULL gross quoteIn, and the unused principal also goes to feeReceiver. A user's buy request that crosses the cap loses (overpaid fees + unused principal).

Recommendations:

Consider to recompute fees against `requiredQuoteIn` after clamping, refund (`excessQuoteIn` + overpaid fees) to the buyer and not `feeReceiver`.

Nad.Fun: Acknowledged. User-facing buys are expected to go through `NadFunRouter`. The router pre-quotes the capped output, computes the required input with `getAmountIn`, and only transfers the required quote amount into `BondingCurve`, leaving/refunding excess at the router/user layer. This behavior is covered by the near-graduation router tests for ERC20 and native buys.

4.2 Low Risk

A total of 13 low risk findings were identified.

[L-1] buy reverts instead of charging fee when totalFeeRate >= BPS

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [BondingCurve.sol#L447-L450](#)
- [BondingCurveLibrary.sol#L19](#)

Description:

In `_calculateFees`, when `totalFeeRate ≥ BPS`, the entire input amount is consumed by fees (sniping + protocol), and `amountAfterFees` is intentionally left as 0 to reflect that no amount remains for the trade:

```
function _calculateFees(address token, uint256 amount,
Curve storage curve, bool withSniping)
    internal
    view
    returns (uint256 protocolFee, uint256 snipingFee,
uint256 creatorFee, uint256 amountAfterFees)
{
    ...

    if (totalFeeRate < BPS) {
        ...
    } else {
        snipingFee = FixedPointMathLib.mulDivUp(amount, snipingFeeRate,
BPS);
        protocolFee = amount - snipingFee;
    }
}
```

In `_buyV1`, this zero value is passed directly to `BondingCurveLibrary.getAmountOut` as `amountIn`:

```
function _buyV1(address to, address token, Curve storage curve)
internal returns (uint256 tokenOut) {
    ...
    (uint256 protocolFee, uint256 snipingFee, uint256 creatorFee,
uint256 quoteInAfterFees) =
        _calculateFees(token, quoteIn, curve, true);
    tokenOut = BondingCurveLibrary.getAmountOut(
        quoteInAfterFees, curve.k, curve.virtualQuoteReserve,
        curve.virtualTokenReserve
    );
    ...
}
```

`BondingCurveLibrary.getAmountOut` enforces `amountIn > 0` and reverts:

```
function getAmountOut(uint256 amountIn, uint256 k, uint256 reserveIn,
uint256 reserveOut)
internal
pure
returns (uint256 amountOut)
{
    require(amountIn > 0 && reserveIn > 0 && reserveOut > 0, "Invalid
inputs");

    uint256 newReserveIn = reserveIn + amountIn;

    uint256 newReserveOut = FixedPointMathLib.mulDivUp(k, 1,
newReserveIn);

    require(newReserveOut < reserveOut, "Insufficient liquidity");

    amountOut = reserveOut - newReserveOut;
}
```

As a result, the entire buy transaction reverts rather than collecting the intended penalty fees. This defeats the purpose of the anti-sniping mechanism: instead of penalizing snipers by taking all their funds as fees, the transaction simply fails and they lose nothing.

Recommendations:

When `amountAfterFees == 0`, `_buyV1` should skip the curve calculation entirely, distribute the fees, and return early with `tokenOut = 0`.

Nad.fun: Resolved with [@89d2b97 ...](#)

Zenith: Verified.

[L-2] Creator fee is charged in initial buy

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L430](#)

Description:

In BondingCurve.create, the docs mentions: If extra quote detected: _initialBuy (sniping-free, protocol fee only)

However _initialBuy use _calculateFees(token, quoteIn, curve, false); and still charge the creatorFee

```
function _calculateFees(address token, uint256 amount,
Curve storage curve, bool withSniping)
    internal
    view
    returns (uint256 protocolFee, uint256 snipingFee,
uint256 creatorFee, uint256 amountAfterFees)
{
    ...
    uint256 snipingFeeRate = withSniping ? _getSnipingFeeRate(token) :
0;
    uint256 protocolFeeRate
= _protocolManager.curveProtocolFeeRate(curve.quoteToken);
    uint256 creatorFeeRate = curve.creatorFeeRate;
    uint256 totalFeeRate = snipingFeeRate + protocolFeeRate
+ creatorFeeRate;
    ...
}
```

Recommendations:

It's recommended to skip the creatorFeeRate also if the intention is protocol fee only

Nad.fun: We acknowledge the wording mismatch, but no contract change is planned

because charging the creator fee on the initial buy matches the protocol economics.

[L-3] Protocol does not support fee-on-transfer quote tokens

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L410](#)
- [CreatorFeeProcessor.sol#L55](#)

Description:

If a fee-on-transfer token is configured as a quote token, two flows will break:

- `_graduateV1`: `quoteBalanceAfterGraduateFee` is transferred to `LPManager`, but the actual received amount is less due to transfer fees. When `LPManager._addLiquidityV2` subsequently attempts to use the original `quoteBalanceAfterGraduateFee` amount, the call reverts.

```
function _graduateV1(address token, Curve storage curve) internal {
    ...

    IERC20(token).safeTransfer(lpManager, tokenForLiquidity);
    ▷ IERC20(curve.quoteToken).safeTransfer(lpManager,
        quoteBalanceAfterGraduateFee);

    ILPManager(lpManager)
        .addLiquidity(
            token, curve.quoteToken, tokenForLiquidity,
            quoteBalanceAfterGraduateFee, curve.dexType, curve.pair
        );

    emit Graduate(token, curve.pair);
}
```

- `CreatorFeeProcessor.processCreatorFee`: The contract pulls amount from `FeeCollector`, but receives less after the transfer fee. The subsequent `_distributeToVaults` call using the original amount will revert.

```
function processCreatorFee(address token, address quoteToken,
    uint256 amount) external {
```

```
        if (msg.sender ≠ feeCollector) revert NotAuthorized();  
        if (amount = 0) return;  
  
        ▷ IERC20(quoteToken).safeTransferFrom(msg.sender, address(this),  
        amount);  
  
        _distributeToVaults(token, quoteToken, amount);  
    }
```

Recommendations:

Do not register fee-on-transfer tokens as quote tokens.

Nad.fun: Acknowledged. We will only register standard ERC20 tokens without transfer fees or rebasing behavior. No code change is planned for this item.

[L-4] virtualTokenReserve is not bounded

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [Token.sol#L18](#)

Description:

The token supply is fixed at 1_000_000_000 ether:

```
contract Token is ERC20Upgradeable, ERC20PermitUpgradeable, IToken {
    uint256 public constant TOTAL_SUPPLY = 1_000_000_000 ether;
```

However, addQuoteToken imposes no upper bound on the virtualTokenReserve parameter:

```
function addQuoteToken(
    address quoteToken,
    uint256 virtualReserve,
    uint256 virtualTokenReserve,
    uint256 minTokenReserve,
    uint256 deployFee_,
    uint256 graduateFee_,
    uint16 curveProtocolFeeRate_,
    uint16 dexProtocolFeeRate_,
    uint256 settlementThreshold_
) external onlyOwner {
    if (quoteToken == address(0)) revert ZeroAddress();
    if (_configs[quoteToken].active) revert QuoteTokenAlreadyAdded();
    require(curveProtocolFeeRate_ <= 1000, "Protocol fee too high");
    require(dexProtocolFeeRate_ <= 1000, "Dex protocol fee too high");

    ...
}
```

During graduation, _graduateV1 computes the token amount needed for DEX liquidity as:

uint256 tokenForLiquidity = quoteBalanceAfterGraduateFee * curve.virtualTokenReserve /

curve.virtualQuoteReserve;

If virtualTokenReserve was set too high such that $\text{tokensSoldToReachGraduation} + \text{tokenForLiquidity} > \text{Token.TOTAL_SUPPLY}$, tokenForLiquidity will exceed currentTokenBalance, causing the subtraction below to underflow and revert:

```
function _graduateV1(address token, Curve storage curve) internal {
    ...

    ▷ uint256 tokenForLiquidity = quoteBalanceAfterGraduateFee
      * curve.virtualTokenReserve / curve.virtualQuoteReserve;

    {
        uint256 currentTokenBalance
    = IERC20(token).balanceOf(address(this));
    ▷ uint256 excessToken = currentTokenBalance - tokenForLiquidity;
      if (excessToken > 0) {
          IERC20(token).safeTransfer(_protocolManager.feeReceiver(),
excessToken);
      }
    }

    ...
}
```

Because graduation is triggered automatically and there is no recovery path, the token and its accumulated quote balance become stuck.

Recommendations:

In addQuoteToken, enforce the graduation-balance invariant $\text{tokensSoldToReachGraduation} + \text{tokenForLiquidity} \leq \text{Token.TOTAL_SUPPLY}$ directly so misconfigurations are rejected at registration time.

Nad.Fun: Resolved with [@2957f5f ...](#)

Zenith: Verified.

[L-5] No slippage protection in vault swaps during settlement

SEVERITY: Low**IMPACT:** Medium**STATUS:** Resolved**LIKELIHOOD:** Low

Target

- [FeeCollector.sol#L131](#)

Description:

When `FeeCollector.settle` is called, it forwards accumulated creator fees to `CreatorFeeProcessor`, which distributes them to registered vaults. `BurnVault`, `GiftVault` and `LPVault` then execute swaps directly against `NadFunPair` or `BondingCurve` with no slippage parameters, making them vulnerable to sandwich attacks.

```
▷ function settle(address pair) external restricted {  
    ...  
}
```

While settlement is restricted to authorized operators, limiting the attacker's ability to time the call, settlement transactions are still visible in the local mempool and can be front-run. A sandwicher (which is a Monad validator) can manipulate the `NadFunPair` price before the vault swap executes, causing the vault to receive significantly fewer tokens, and then profit on the backrun (if the received amount is more than swap fees). The value lost comes directly from creator fee distributions.

Recommendations:

Each vault should independently handle its own slippage protection when interacting with the bonding curve or `NadFunPair` pools. Each vault's swap logic should accept a minimum output amount seting externally by the caller.

Nad.Fun: Resolved with [@e74c29a742 ...](#)

Zenith: Verified.

[L-6] getAmountIn does not enforce exact out semantics

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [BondingCurve.sol#L520-L523](#)

Description:

getAmountIn is intended to quote the exact input required to receive a specified amountOut. However, when amountOut exceeds availableTokenOut, the function silently clamps it to availableTokenOut and returns the amountIn required for that reduced amount instead of reverting.

```
function getAmountIn(address token, uint256 amountOut, bool isBuy)
external view returns (uint256 amountIn) {
    ...

    if (isBuy) {
        uint256 availableTokenOut = curve.virtualTokenReserve
        - curve.minTokenReserve;
        if (amountOut > availableTokenOut) {
            amountOut = availableTokenOut;
        }

        uint256 quoteInAfterFees = BondingCurveLibrary.getAmountIn(
            amountOut, curve.k, curve.virtualQuoteReserve,
            curve.virtualTokenReserve
        );
        amountIn = totalFeeRate == 0
            ? quoteInAfterFees
            : FixedPointMathLib.mulDivUp(quoteInAfterFees, BPS, BPS
        - totalFeeRate);
    } else {
        ...
    }
}
```

This breaks callers that rely on the exact-out guarantee. The router may still revert after execution because it checks the actual output, but external callers using `getAmountIn` as an exact-out quote can be misled.

Recommendations:

Revert with an explicit error (e.g. `InsufficientTokenOut`) when the requested `amountOut` exceeds `availableTokenOut`, rather than silently capping it.

Nad.Fun: Resolved with [@2b4e31d ...](#)

Zenith: Verified.

[L-7] Graduation can revert if graduateFee is increased post creation

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L393](#)

Description:

graduateV1 fetches graduateFee live from ProtocolManager at the time of graduation rather than using the value set when the token was created. If the protocol admin raises graduateFee after a token is deployed such that it exceeds the curve's quoteBalanceBeforeGraduateFee, the require check will revert, bricking graduation for that token.

```
function _graduateV1(address token, Curve storage curve) internal {
    ...
    {
        ▶ uint256 graduateFee_
        = _protocolManager.graduateFee(curve.quoteToken);
        if (graduateFee_ > 0) {
            ▶ require(quoteBalanceBeforeGraduateFee > graduateFee_,
                "Insufficient for graduate fee");
                quoteBalanceAfterGraduateFee = quoteBalanceBeforeGraduateFee
                - graduateFee_;
                IERC20(curve.quoteToken).safeTransfer(_protocolManager.feeReceiver(),
                graduateFee_);
            }
        }
        ...
    }
}
```

Recommendations:

Snapshot `graduateFee` into the curve's state at token creation time and use the stored value during graduation. Also checking that `quoteBalanceBeforeGraduateFee > graduateFee_` at graduation when creating token.

This ensures that fee increases only affect newly created tokens and cannot retroactively block existing curves from graduating.

Nad.Fun: Resolved with [@f476369 ...](#)

Zenith: Verified.

[L-8] Incorrect seller in Sell Event

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L337](#)

Description:

`_sellV1` emits the `Sell` event with `msg.sender` as the seller. When calls are routed through `NadFunRouter`, `msg.sender` resolves to the router's address rather than the actual user, causing the event to log an incorrect seller.

```
event Sell(address indexed token, address indexed seller,
uint256 tokenIn, uint256 quoteOut);

function _sellV1(address to, address token, Curve storage curve)
internal returns (uint256 quoteOutAfterFees) {
    ...
    emit Sell(token, msg.sender, tokenIn, quoteOutAfterFees);
}
```

Recommendations:

Pass an explicit seller into `BondingCurve.sell`.

Nad.Fun: Resolved with [@329491e ...](#)

Zenith: Verified.

[L-9] Duplicate vault addresses are not rejected in _setupVaults

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L183-L190](#)

Description:

_setupVaults does not validate that each vault address in the allocations array is unique. A creator can therefore supply the same vault address in multiple entries, causing IVault.setup() to be called on that vault more than once per token and splitting BPS weight across duplicate slots.

```
function _setupVaults(address token, VaultAllocation[]  
calldata allocations)  
    internal  
    returns (ICreatorFeeProcessor.VaultSlot[] memory vaultSlots)  
{  
    ...  
  
    vaultSlots  
    = new ICreatorFeeProcessor.VaultSlot[](allocations.length);  
  
    > for (uint256 i = 0; i < allocations.length; i++) {  
        address vault = allocations[i].vault;  
        require(IVaultRegistry(registry).isActive(vault), "Vault not  
active");  
  
        IVault(vault).setup(token, allocations[i].setupData);  
  
        vaultSlots[i] = ICreatorFeeProcessor.VaultSlot({vault: vault,  
bps: allocations[i].bps});  
    }  
}
```

It means creator fee distribution sends multiple separate transfers to the same vault instead of one consolidated transfer.

Recommendations:

- Add a duplicate-address check inside the loop OR
- Every vault should check if the token is already registered.

Nad.fun: Resolved with [@27cd2ddd9d ...](#)

Zenith: Verified.

[L-10] BondingCurve `_calculateFees` zeroes `amountAfterFees` when `totalFeeRate` $\geq$ BPS, even if `snipingFeeRate` + `protocolFeeRate` < BPS

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L449](#)

Description:

`BondingCurve._calculateFees()` has a special-case branch for configurations where the aggregate buy-side fee rate is greater than or equal to BPS:

```
if (totalFeeRate < BPS) {
    // ...
} else {
    snipingFee = FixedPointMathLib.mulDivUp(amount, snipingFeeRate, BPS);
    protocolFee = amount - snipingFee;
    // @audit - amountAfterFees stays 0
}
```

The problem is that `amountAfterFees` is never assigned in the `else` branch.

This is incorrect when `snipingFeeRate` + `protocolFeeRate` < BPS but `snipingFeeRate` + `protocolFeeRate` + `creatorFeeRate` $\geq$ BPS

In that scenario, there is still positive quote available for the trade after sniping and protocol fees. A reasonable saturation policy would clip or suppress the creator fee and leave a non-zero `amountAfterFees`. Instead, the current code sets the effective trade input to zero and causes the downstream buy path to revert.

In `_buyV1()` the returned `quoteInAfterFees` is passed directly into `BondingCurveLibrary.getAmountOut()`:

```
tokenOut = BondingCurveLibrary.getAmountOut(
    quoteInAfterFees, curve.k, curve.virtualQuoteReserve,
    curve.virtualTokenReserve
);
```

Since `quoteInAfterFees = 0`, `getAmountOut()` reverts with:

```
require(amountIn > 0 && reserveIn > 0 && reserveOut > 0, "Invalid inputs");
```

Recommendations:

The saturation branch should explicitly preserve residual trade input whenever `snipingFee + protocolFee < amount`.

Nad.fun: Resolved with [@ac90327684 ...](#)

Zenith: Verified.

[L-11] NadFunFactory.createPair permissionless, pre-deploy DOS for predicted salt

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [NadFunFactory.sol#L32-L48](#)

Description:

NadFunFactory.createPair is external with no access control. An attacker can monitor the mempool for BondingCurve.create() transactions, compute the deterministic clone address from params.salt + bytecode, and pre-call factory.createPair(predictedToken, params.quoteToken). The legitimate BondingCurve._deployPairViaFactory then reverts with PairExists → entire create reverts.

```
function createPair(address tokenA, address tokenB)
external returns (address pair) {
    if (tokenA == tokenB) revert IdenticalAddresses();
    (address token0, address token1) = tokenA < tokenB ? (tokenA, tokenB)
: (tokenB, tokenA);
    if (token0 == address(0)) revert ZeroAddress();
    if (getPair[token0][token1] != address(0)) revert PairExists();

    bytes32 salt = keccak256(abi.encodePacked(token0, token1));
    pair = Clones.cloneDeterministic(implementation, salt);

    NadFunPair(pair).initialize(address(this), token0, token1,
feeCollector);

    getPair[token0][token1] = pair;
    getPair[token1][token0] = pair;
    allPairs.push(pair);

    emit PairCreated(token0, token1, pair, allPairs.length);
}
```

This can cause sustained DOS on token launches.

Recommendations:

Consider restrict `createPair` to `onlyBondingCurve`.

Nad.fun: Acknowledged. The described griefing requires the attacker to know the launch salt/predicted token address before creation; under our deployment flow the salt is not intended to be disclosed before submission, and public mempool observation is not assumed to be generally available on Monad.

[L-12] BondingCurve setModule enables retroactive re-routing of all fees

SEVERITY: Low

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L548-L551](#)

Description:

The `setModule` function rebinds any of six module IDs to any address:

```
function setModule(bytes32 moduleId, address module)
    external onlyRole(DEFAULT_ADMIN_ROLE) {
        _modules[moduleId] = module;
        emit ModuleUpdate(moduleId, module);
    }
```

Modules are reads at every consuming function call, `_modules[MODULE_FEE_COLLECTOR]`, `_modules[MODULE_FACTORY]`, `_modules[MODULE_LP_MANAGER]` etc, causing all in-flight trades silently switch logic when admin swaps a module.

If not done carefully, swapping `MODULE_FEE_COLLECTOR` strands the old `FeeCollector`'s `_trackedBalance` and `_accumulatedFees` mappings and breaks pre-existing pairs which still hold the old `FeeCollector` address.

Recommendations:

When switching modules, ensure that all state remains consistent to avoid breaking accounting.

Nad.fun: Resolved with (@ad4ee5c ...)[<https://github.com/Naddotfun-nadfun-contract-v2/Naddotfun-nadfun-contract-v2/commit/ad4ee5c>]

Zenith: Verified.

[L-13] BondingCurve _calculateFees else-branch zeroes creator fee, but FeeCollector re-splits via creator-rate share

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [BondingCurve.sol#L447-L450](#)
- [FeeCollector.sol#L102-L121](#)

Description:

The `_calculateFees` function has two branches. When `totalFeeRate < BPS`, it computes both `protocolFee` and `creatorFee` proportionally. When `totalFeeRate ≥ BPS`, the else-branch sets:

```
// ...
} else {
    snipingFee = FixedPointMathLib.mulDivUp(amount, snipingFeeRate, BPS);
    protocolFee = amount - snipingFee;
    // @audit - creatorFee never assigned
}
// ...
```

`_sendCombinedFee` then transfers `combinedFee = protocolFee + creatorFee = protocolFee + 0` to `FeeCollector`. `FeeCollector`'s `collectFee` re-splits this received amount based on its own per-pair `creatorFeeRate` and `activeProtocolFeeRate`, independently of what `BondingCurve` labelled the fee:

```
// ...
> uint256 protocolFee = FixedPointMathLib.mulDivUp(feeReceived,
    uint256(activeProtocolFeeRate), combinedFeeRate);
> uint256 creatorFee = feeReceived - protocolFee;

    if (protocolFee > 0) {
        IERC20(quoteTokenAddr)
            .safeTransfer(IProtocolManager(authority()).feeReceiver(),
                protocolFee);
    }
```



```
        if (creatorFee > 0) {  
            _accumulatedFees[pair] += creatorFee;  
        }  
    }  
    // ...
```

With default config (creatorFeeRate=500, curveProtocolFeeRate=100), an amount = 1 ether buy in the sniping window:

- For instance, snipingFee = 0.99 ether → feeReceiver
- protocolFee (BondingCurve's view) = 0.01 ether → FeeCollector
- FeeCollector splits 0.01 ether: ~0.00167 ether to feeReceiver, ~0.00833 ether to creator vault accumulator.

So creator vaults systematically receive ~83% of what BondingCurve classified as 100% protocol fee.

Creator vaults accumulate funds that the BondingCurve's fee-classification logic intended for the protocol.

Recommendations:

Consider to align the two contracts on a single fee-classification model (BondingCurve forwards protocolFee and creatorFee separately, FeeCollector trusts the labels and accumulates without re-splitting).

Nad.Fun: Resolved with [@7cd50802c2 ...](#)

Zenith: Verified.

4.3 Informational

A total of 13 informational findings were identified.

[I-1] Missing aggregate fee cap can brick NadFunPair quoting and DEX swap execution

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [NadFunPair.sol#L285](#)
- [NadFunPair.sol#L245](#)

Description:

NadFunPair assumes that the aggregate Nad fee, $\text{feeRate} = \text{creatorFeeRate} + \text{dexProtocolFeeRate}$ is strictly less than $\text{BPS} - \text{LP_FEE_RATE} = 10000 - 25 = 9975$. but this invariant is never enforced.

```
function _collectFee(uint256 amount0In, uint256 amount1In,
uint256 amount0Out, uint256 amount1Out) private {
    address feeCollectorAddr = feeCollector;
    try IFeeCollector(feeCollectorAddr).getFeeConfig(address(this))
returns (
    IFeeCollector.FeeConfig memory config
) {
    uint16 feeRate = config.creatorFeeRate
+ config.dexProtocolFeeRate;
    address quoteToken_ = config.quoteToken;
    if (feeRate == 0) return;

    bool quoteIsToken0 = (quoteToken_ == token0);
    uint256 swapFee;

    uint256 quoteIn = quoteIsToken0 ? amount0In : amount1In;
    if (quoteIn > 0) {
        swapFee += quoteIn * uint256(feeRate) / BPS;
```

```
    }  
  
    uint256 quoteOut = quoteIsToken0 ? amount0Out : amount1Out;  
    if (quoteOut > 0) {  
        swapFee += quoteOut * uint256(feeRate) / (BPS - LP_FEE_RATE  
        - feeRate);  
    }  
  
    if (swapFee > 0) {  
        _safeTransfer(quoteToken_, feeCollectorAddr, swapFee);  
        IFeeCollector(feeCollectorAddr).collectFee(address(this));  
    }  
    } catch {}  
}
```

The fee components are configured across two different places:

- creatorFeeRate can be allowlisted at arbitrary uint16 values via `ProtocolManager.setAllowedCreatorFeeRates()`
- dexProtocolFeeRate is individually capped at 1000, but there is no aggregate check against creatorFeeRate

As a result, a pair can be configured such that $\text{creatorFeeRate} + \text{dexProtocolFeeRate} \geq 9975$.

Recommendations:

Consider enforce the aggregate invariant anywhere fee configuration can change.

Nad.fun: Resolved with [@242db17da6...](#)

Zenith: Verified.

[I-2] Direct use of BondingCurve.buy and BondingCurve.sell exposes users to sandwich attacks

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L223](#)
- [BondingCurve.sol#L308](#)

Description:

BondingCurve.buy and BondingCurve.sell perform no slippage checks, users who call them directly have no protection against sandwich attacks. Slippage validation is only performed at the NadFunRouter layer.

Recommendations:

Add NatSpec documentation to BondingCurve.buy and BondingCurve.sell explicitly warning that these functions do not provide slippage protection and should only be called via NadFunRouter.

Nad.Fun: Resolved with [@f3245c39f1 ...](#)

Zenith: Verified.

[I-3] Misleading comment in BondingCurveLibrary

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurveLibrary.sol#L7](#)

Description:

BondingCurveLibrary contains comments claiming that FixedPointMathLib.mulDivUp performs 512-bit safe ceiling division (e.g., "Uses mulDivUp(k, 1, newReserveIn) for 512-bit safe ceiling division"). This is inaccurate. As shown in the Solady source, mulDivUp operates on standard 256-bit arithmetic and reverts on overflow.

```
/// @dev Returns `ceil(x * y / d)`.
/// Reverts if `x * y` overflows, or `d` is zero.
function mulDivUp(uint256 x, uint256 y, uint256 d)
    internal pure returns (uint256 z) {
    /// @solidity memory-safe-assembly
    assembly {
        z := mul(x, y)
        // Equivalent to `require(d != 0 && (y == 0 || x <= type(uint256).max / y))`.
        if iszero(mul(or(iszero(x), eq(div(z, x), y)), d)) {
            mstore(0x00, 0xad251c27) // `MulDivFailed()`
            revert(0x1c, 0x04)
        }
        z := add(iszero(iszero(mod(z, d))), div(z, d))
    }
}
```

Recommendations:

Remove or correct the misleading comment to accurately reflect that mulDivUp reverts on overflow rather than using 512-bit arithmetic.

Nad.Fun: Resolved with [@c2c06db ...](#)

Zenith: Verified.

[I-4] Token address is not validated in `getAmountIn` and `getAmountOut`

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [NadFunPair.sol#L201-L251](#)

Description:

`getAmountIn` derives `tokenIn` by assuming any address that is not `token0` must be `token1`. This means if an arbitrary address is passed as `tokenOut`, the function silently proceeds using `token1` as `tokenIn` rather than reverting.

```
function getAmountIn(address tokenOut, uint256 amountOut)
external view returns (uint256 amountIn) {
    address tokenIn = tokenOut == token0 ? token1 : token0;
    (uint256 reserveIn, uint256 reserveOut) =
▷      tokenIn == token0 ? (uint256(_reserve0), uint256(_reserve1)) :
      (uint256(_reserve1), uint256(_reserve0));
    ...
}
```

The same issue exists in `getAmountOut`, where `tokenIn` is used to select reserves without first verifying it matches either `token0` or `token1`:

```
function getAmountOut(address tokenIn, uint256 amountIn)
external view returns (uint256 amountOut) {
    (uint256 reserveIn, uint256 reserveOut) =
▷      tokenIn == token0 ? (uint256(_reserve0), uint256(_reserve1)) :
      (uint256(_reserve1), uint256(_reserve0));
    ...
}
```

Recommendations:

Add explicit validation that the provided token address matches one of the two pair tokens, and revert otherwise.

Nad.Fun: Resolved with [@5e3517b ...](#)

Zenith: Verified.

[I-5] _mintFee preserves a 1/6 LP-fee share despite the reduced 0.25% LP fee

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [NadFunPair.sol#L24](#)

Description:

The `_mintFee` function mints LP tokens to the protocol as a fee, using the Uniswap v2 formula where the denominator $\text{rootK} * 5 + \text{rootKLast}$ encodes a 1/6 share of LP fees for the protocol. In Uniswap v2, this yields a 0.05% protocol fee on a 0.30% LP fee ($0.30\% \times 1/6 = 0.05\%$).

However, NadFunPair sets `LP_FEE_RATE = 25` (0.25%). With the unchanged denominator, the protocol receives only $0.25\% \times 1/6 \approx 0.042\%$, not the 0.05%.

```
uint256 public constant LP_FEE_RATE = 25;

function _mintFee(uint112 reserve0, uint112 reserve1)
    private returns (bool feeOn) {
    address feeTo = INadFunFactoryMinimal(factory).feeTo();
    feeOn = feeTo != address(0);
    uint256 kLast_ = kLast;
    if (feeOn) {
        if (kLast_ != 0) {
            uint256 rootK = Math.sqrt(uint256(reserve0) * uint256(reserve1));
            uint256 rootKLast = Math.sqrt(kLast_);
            if (rootK > rootKLast) {
                uint256 numerator = totalSupply() * (rootK - rootKLast);
                uint256 denominator = rootK * 5 + rootKLast;
                uint256 liquidity = numerator / denominator;
                if (liquidity > 0) _mint(feeTo, liquidity);
            }
        }
    } else if (kLast_ != 0) {
        kLast = 0;
    }
}
```

```
}  
}
```

Recommendations:

Decide on the intended protocol fee. If the target is 0.05%, update the denominator to `_mintFee` to reflect the correct 1/5 share at the 0.25% LP fee rate. If the intended design is a 1/6 share of LP fees, no change is needed.

Nad.fun: Resolved with [@b642c2e ...](#)

Zenith: Verified.

[I-6] getAmountOut silently returns 0 instead of reverting

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L495](#)

Description:

`BondingCurve.getAmountOut` silently returns 0 when `quoteInAfterFees == 0`, while the actual `_buyV1` execution path causes `BondingCurveLibrary.getAmountOut` to revert under the same condition. This inconsistency means callers relying on `getAmountOut` as a pre-flight simulation will receive a misleading 0 rather than an indication that the transaction would revert.

```
function getAmountOut(address token, uint256 amountIn, bool isBuy)
external view returns (uint256 amountOut) {
    Curve storage curve = curves[token];

    if (isBuy) {
        (,,, uint256 quoteInAfterFees) = _calculateFees(token, amountIn,
curve, true);
        if (quoteInAfterFees == 0) return 0;
        ...
    }
}
```

Recommendations:

Replace the early return with a revert to match the behaviour of the actual buy path.

Nad.Fun: Resolved with [@50f1688f0e...](#)

Zenith: Verified.

[I-7] `_calculateFees` can revert when `snipingFeeRate` exceeds 100%

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L447-L450](#)

Description:

`setSnipingPenaltyConfig` does not validate that the resulting sniping fee rate stays below 100%:

```
function setSnipingPenaltyConfig(uint256 duration, uint16 penaltyPerMinute)
    external onlyOwner {
        _snipingDuration = duration;
        _snipingPenaltyPerMinute = penaltyPerMinute;
        emit SnipingPenaltyUpdate(duration, penaltyPerMinute);
    }
```

If `snipingFeeRate` exceeds BPS (> 100%), `_calculateFees` takes the else branch where `snipingFee` can exceed amount, causing an underflow revert on the subtraction:

```
function _calculateFees(address token, uint256 amount,
    Curve storage curve, bool withSniping)
    internal
    view
    returns (uint256 protocolFee, uint256 snipingFee,
    uint256 creatorFee, uint256 amountAfterFees)
{
    ...
} else {
    snipingFee = FixedPointMathLib.mulDivUp(amount, snipingFeeRate,
    BPS);
    protocolFee = amount - snipingFee;
}
```

This would cause buy and sell calls to revert for affected tokens during the sniping window, effectively halting trading.

Recommendations:

- Add a validation check in `setSnipingPenaltyConfig` to ensure the maximum possible `snipingFeeRate` does not exceed BPS.
- Alternatively, cap `protocolFee` at zero instead of allowing an underflow.

Nad.fun: Resolved with [@bb36f0e ...](#)

Zenith: Verified.

[I-8] Dead code in `_calculateFees`

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L441](#)

Description:

Inside the `totalFeeRate < BPS` branch of `_calculateFees`, since `snipingFeeRate` is a component of `totalFeeRate`, `snipingFee ≤ totalFee` is always true. The guard `if (snipingFee > totalFee)` can never be triggered and constitutes unreachable dead code.

```
function _calculateFees(address token, uint256 amount,
Curve storage curve, bool withSniping)
    internal
    view
    returns (uint256 protocolFee, uint256 snipingFee,
uint256 creatorFee, uint256 amountAfterFees)
{
    ...
    if (totalFeeRate < BPS) {
        uint256 totalFee = FixedPointMathLib.mulDivUp(amount,
totalFeeRate, BPS);
        amountAfterFees = amount - totalFee;
        snipingFee = FixedPointMathLib.mulDivUp(amount, snipingFeeRate,
BPS);
        if (snipingFee > totalFee) snipingFee = totalFee;
        uint256 remainingFee = totalFee - snipingFee;
        protocolFee = FixedPointMathLib.mulDivUp(amount,
protocolFeeRate, BPS);
        if (protocolFee > remainingFee) protocolFee = remainingFee;
        remainingFee -= protocolFee;
        creatorFee = remainingFee;
    } else {
        ...
    }
}
```

Recommendations:

Remove the unreachable guard `if (snipingFee > totalFee) snipingFee = totalFee;.`

Nad.fun: Resolved with [@eabb95575b...](#)

Zenith: Verified.

[I-9] BondingCurve getAmountOut / getAmountIn continue to quote after graduation

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L490-L538](#)

Description:

After a token has graduated, BondingCurve.buy() and BondingCurve.sell() correctly reject further curve trading:

```
// ...  
if (curve.graduated) revert AlreadyGraduated();  
// ...
```

However, the corresponding quote functions do not enforce the same state boundary:

```
function getAmountOut(address token, uint256 amountIn, bool isBuy)  
    external view returns (uint256 amountOut) {  
    Curve storage curve = curves[token];  
    // no graduation check  
    // ...  
}  
  
function getAmountIn(address token, uint256 amountOut, bool isBuy)  
    external view returns (uint256 amountIn) {  
    Curve storage curve = curves[token];  
    // no graduation check  
    // ...  
}
```

As a result, once `curve.graduated = true`, the quote functions operate on the frozen terminal bonding-curve reserves instead of reporting that the curve is no longer tradable.

Recommendations:

Consider to revert once curve is graduated.

Nad.Fun: Resolved with [@25aef90eb4 ...](#)

Zenith: Verified.

[I-10] Missing zero address check for MODULE_CREATOR_FEE_PROCESSOR

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L154](#)

Description:

`_create` does not validate that `MODULE_CREATOR_FEE_PROCESSOR` has been set before use. If `_modules[MODULE_CREATOR_FEE_PROCESSOR]` returns `address(0)`, the subsequent call to `ICreatorFeeProcessor(processor).setup(...)` will revert with no informative error, inconsistent with how other modules are guarded elsewhere in the codebase.

```
function _create(CreateTokenParams calldata params, address creator)
internal returns (address token) {
    ...
    ICreatorFeeProcessor.VaultSlot[] memory vaultSlots
= _setupVaults(token, params.vaults);
    address processor = _modules[MODULE_CREATOR_FEE_PROCESSOR];
    ICreatorFeeProcessor(processor).setup(token, vaultSlots);
    ...
}
```

Recommendations:

Add an explicit zero address check for processor and revert with a descriptive error (e.g., `ModuleNotSet`).

Nad.fun: Resolved with [@e7952e36c2...](#)

Zenith: Verified.

[\[1-1\]](#) [NadFunFactory.setImplementation/setFeeTo](#) and [FeeCollector.setCurveProtocolFeeRate/setDexProtocolFeeRate](#) emit no events

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [NadFunFactory.sol#L50-L59](#)
- [ProtocolManager.sol#L235-L241](#)
- [FeeCollector.sol#L173-L186](#)

Description:

These important parameter changes occur without emitting any event:

```
function setImplementation(address _implementation) external {
    if (msg.sender != protocolManager) revert Forbidden();
    if (_implementation == address(0)) revert ZeroAddress();
    implementation = _implementation;
}

function setFeeTo(address _feeTo) external {
    if (msg.sender != protocolManager) revert Forbidden();
    feeTo = _feeTo;
}

// ...
function setFactoryFeeTo(address factory, address feeTo)
    external onlyOwner {
    INadFunFactory(factory).setFeeTo(feeTo);
}

function setFactoryImplementation(address factory, address impl)
    external onlyOwner {
    INadFunFactory(factory).setImplementation(impl);
}

// ...
function setCurveProtocolFeeRate(address pair, uint16 rate)
    external restricted {
    // ...
```

```
        config.curveProtocolFeeRate = rate;
    }
    function setDexProtocolFeeRate(address pair, uint16 rate)
        external restricted {
        // ...
        config.dexProtocolFeeRate = rate;
    }
```

Recommendations:

Consider emitting events when those functions are called.

Nad.fun: Resolved with [@76d8849dbf...](#)

Zenith: Verified.

[I-12] Donation before `create()` could cause third-party reputational and market-signal manipulation

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [BondingCurve.sol#L104-L107](#)

Description:

`BondingCurve.create()` derives the user's quote-token input via balance-delta accounting:

```
// ...
    uint256 totalIn = IERC20(params.quoteToken).balanceOf(address(this))
    - _totalQuoteReserved[params.quoteToken];
    require(totalIn >= deployFee_, "Insufficient deploy fee");
    ▶ uint256 quoteIn = totalIn - deployFee_;
    if (quoteIn > 0) {
        tokenOut = _initialBuy(creator, token, curves[token],
        quoteIn); // creator = params.creator
    }
// ...
```

`totalIn` is the difference between the contract's current quote-token balance and its internal tracked balance. The protocol's intended pattern is, the router pre-transfers exactly `deployFee + buyQuoteAmount` to `BondingCurve`, then calls `create()`, which sees the delta and processes it.

Because `totalIn` is derived from the actual contract balance, not an explicit parameter, any quote tokens transferred to `BondingCurve` from any source between the previous state-changing call and the `create()` call are absorbed as part of `totalIn`. After the `deployFee_` deduction, the remainder is forwarded to `_initialBuy(creator, ...)` which mints tokens to the creator's wallet.

Creator forced into an oversized initial position they did not budget for. On-chain history permanently shows them as a whale buyer of their own token. Cannot easily exit without dumping. Reputational damage that cannot be undone, the on-chain footprint is permanent.

Recommendations:

Consider allowing the creator to explicitly determine the amount they are buying, then the excess can be transferred to the fee receiver.

Nad.Fun: Resolved with [@2381bff6d2...](#)

Zenith: Verified.

[I-13] addQuoteToken and updateQuoteToken accept zero or inverted reserves

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [ProtocolManager.sol#L117-L158](#)
- [ProtocolManager.sol#L117-L158](#)

Description:

Neither addQuoteToken nor updateQuoteToken validates the relationships between virtualReserve, virtualTokenReserve, and minTokenReserve. All three are forwarded to BondingCurve._initCurve and stored verbatim:

- $\text{virtualReserve} = 0 \rightarrow \text{BondingCurveLibrary.getAmountOut}$ reverts with "Invalid inputs" ($\text{require}(\text{reserveIn} > 0)$) on every buy. All tokens created with this quote are permanently buy-DoS'd.
- $\text{virtualTokenReserve} = 0 \rightarrow$ same path.
- $\text{minTokenReserve} \geq \text{virtualTokenReserve} \rightarrow \text{_buyV1}$ underflows at $\text{availableTokenOut} = \text{curve.virtualTokenReserve} - \text{curve.minTokenReserve}$.

Admin misconfiguration during deployment bricks all tokens created against the bad quote configuration with no on-chain recovery

Recommendations:

Consider adding the following check :

```
function addQuoteToken(...) external onlyOwner {
    // ...
    require(virtualReserve > 0, "Zero virtual quote reserve");
    require(virtualTokenReserve > 0, "Zero virtual token reserve");
    require(minTokenReserve < virtualTokenReserve, "minTokenReserve ≥ virtualTokenReserve");
    // ...
}
```

Apply the same checks to `updateQuoteToken`.

Nad.fun: Resolved with [@ed0380e ...](#)

Zenith: Verified.